

Hash-Based Digital Signatures

Lamport OTS $\rightarrow$ WOTS $\rightarrow$ Merkle Signature Scheme

Aicha Slaitane Abdullah Algethami

Applied Cryptography (CS 232) — KAUST

1. Motivation: The Quantum Threat
2. Lamport One-Time Signatures
3. Winternitz OTS (WOTS)
4. Merkle Signature Scheme
5. Hypertrees
5. OTS Benchmarks
6. Key-Reuse Attack
7. MSS Performance Evaluation
8. Parameter Tuning
9. Hypertrees
10. Conclusion

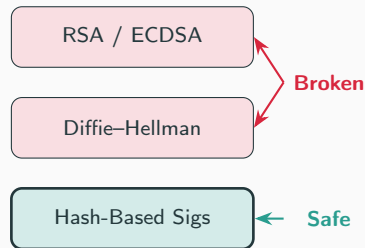
Theory & Implementation

The Quantum Threat

- **Shor's algorithm** (1994) breaks RSA, ECDSA, and Diffie–Hellman in polynomial time on a quantum computer.
- These schemes rely on the hardness of *factoring* or *discrete logs*.
- We need signatures based on different assumptions.

Hash-based signatures rely *only* on the one-wayness of hash functions (e.g. SHA-256).

- No number-theoretic assumptions.
- Grover's algorithm gives only a $\sqrt{}$ speedup $\Rightarrow$ SHA-256 still provides ~ 128 -bit quantum security.



Lamport One-Time Signature

Key Generation: (Where H is a cryptographic collision resistant hash function e.g. SHA-256)

- Generate 256 pairs of random 32-byte blocks.
- Secret key: $SK[i] = (s_i^0, s_i^1)$
- Public key: $PK[i] = (H(s_i^0), H(s_i^1))$

Signing message m :

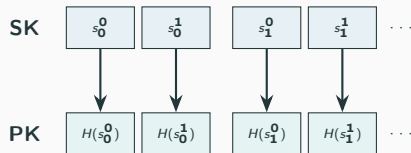
1. Compute $h = H(m)$ — 256 bits.
2. For each bit h_i , reveal $s_i^{h_i}$.

Verification:

Check $H(\sigma_i) \stackrel{?}{=} PK[i][h_i]$ for all i .

Key Properties

Signature = 8,192 B | Signing: 0.03 ms (no hashing!) | **Strictly one-time use**



If $h_0 = 0$: reveal s_0^0

Why Is Lamport One-Time?

- Each signature reveals **256 of 512** secret blocks.
- A second signature on a different message reveals *different* blocks.
- After k reuses, an attacker has collected enough blocks to forge *any* message.

Reuses (k)	Blocks Exposed	Forgery Probability
1	50%	$\approx 0\%$
8	99.6%	38.5%
10	99.9%	78%
15	100%	$>99.9\%$

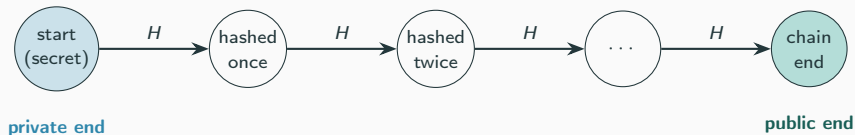
Two solutions: (1) Use WOTS to reduce sizes (2) Use Merkle trees to manage many keys

Winternitz OTS: The Hash Chain

Recall the problem with Lamport: it signs the message *one bit at a time*, so the signature is large (8 KB).

Winternitz idea: sign *several bits at once* as a small number (a “digit”), using a *hash chain*.

A hash chain is one secret value hashed over and over. Each arrow applies H once:

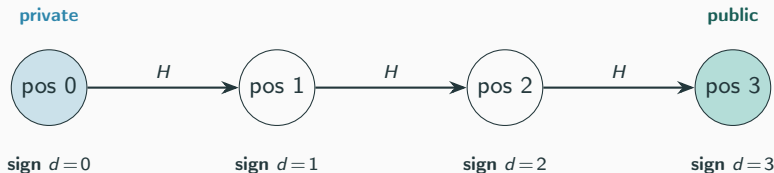


Moving **forward** is easy (just hash). Moving **backward** is infeasible (it would mean inverting H). This one-way direction is what makes the chain secure.

WOTS: Signing One Digit (Toy Example, $w = 4$)

One chain signs **one digit** $d \in \{0, 1, 2, 3\}$. The position along the chain *is* the digit value.

- **Private value:** the start of the chain (random, kept secret).
- **Public value:** the end of the chain — the start hashed $w-1 = 3$ times.
- **To sign digit d :** reveal the value at position d (the start hashed d times).
- **To verify:** hash the revealed value the remaining $3 - d$ times — it must reach the public value.



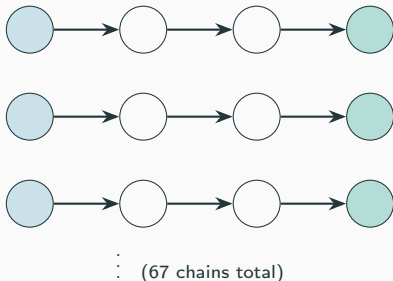
But one chain only signs one digit. A real message hash has many digits — so WOTS uses **many chains in parallel** (next slide).

WOTS: The Full Key is Many Chains

The message hash is split into many digits — and **each digit gets its own chain**. With $w=16$, that is 67 chains side by side.

Secret key

Public key



Secret key = all 67 chain starts.

Public key = all 67 chain ends.

Signature = the middle value of each chain, one per digit.

In one sentence

To sign a message: split its hash into digits, and from each chain reveal the value at that digit's position. The verifier finishes every chain and checks all 67 ends match the public key.

WOTS: Why We Need a Checksum

The problem: hashing *forward* is free for everyone — including the attacker.

- The attacker can hash a revealed value forward to forge a valid value for any *higher* digit.
- Unprotected, they could forge any message whose digits are all $\geq$ ours.

The fix: a checksum $C = \sum_i (w - 1 - d_i)$, signed with extra chains.

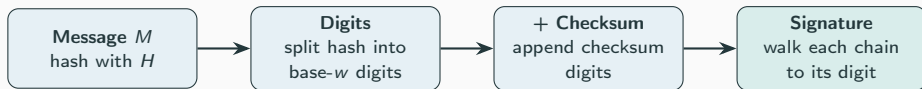
- Raising any message digit makes the checksum *decrease*.
- A smaller checksum digit means going *backward* in a chain — which needs inverting H .
- So the attacker cannot raise a digit without breaking the checksum.

w	Chains (incl. checksum)	Signature Size
4	133	4,256 B
16	67	2,144 B
256	34	1,088 B

Larger $w \Rightarrow$ fewer chains $\Rightarrow$ smaller signature (but longer chains to compute).

WOTS: The Full Picture

Signing a message is just this pipeline — hash it, split into digits, walk each chain:



The three keys, defined

Secret key — one random start value per chain.

Public key — each start hashed $w-1$ times (every chain end).

Signature — each chain stopped at its digit's position.

Verification

Finish every chain from the signature value; all chain ends must equal the public key.

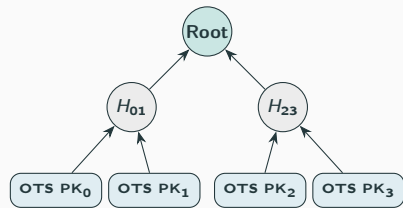
Trade-off: one-time only, but signatures are $4\times$ smaller than Lamport.

From OTS/WOTS to the Merkle Signature Scheme

Both Lamport and WOTS are one-time — a fresh keypair is needed for every message.

Merkle's fix: pre-generate 2^h OTS keypairs and bind them under a single public key using a hash tree.

- Each OTS public key PK_i is hashed to form **leaf** L_i of the tree.
- Internal nodes are computed as $H(\text{left}||\text{right})$.
- The **tree root** is the single MSS public key (32 B).
- To sign message i : run OTS_i and attach the auth path (h sibling nodes) to let anyone recompute the root.



Lamport or WOTS public keys as leaves

Advantage of WOTS over Lamport

WOTS ($w=16$) signatures are $\approx 4\times$ smaller (2 144 B vs. 8 192 B), directly shrinking the on-wire MSS signature.

Merkle Signature Scheme: Key Generation

Problem: One-Time Signatures (OTS) can only sign a single message securely.

Solution: Build a binary hash tree (Merkle Tree) over 2^h OTS public keys to sign 2^h messages.

Key Generation Process:

1. Generate 2^h independent OTS keypairs.
2. Hash each OTS public key to form the leaves: $\text{Leaf}_i = H(\text{OTS_PK}_i)$.
3. Compute parent nodes by hashing the concatenation of children: $\text{Parent} = H(\text{left}||\text{right})$.
4. The **MSS public key** is the tree root (just 32 B).

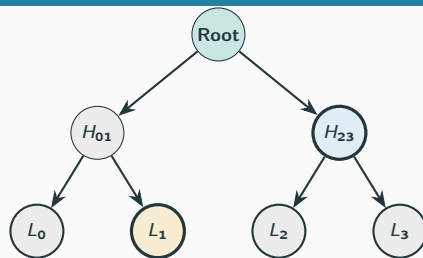
Merkle Signature Scheme: Sign & Verify

To sign the i -th message:

- Sign the message with OTS key i .
- Include the *auth path*: the h sibling hashes needed to reach the root.

To verify:

- Verify the underlying OTS signature against OTS_PK_i .
- Reconstruct the root from Leaf_i and the auth path.
- Check if reconstructed root $\stackrel{?}{=}$ MSS public key.



• Signing leaf

• Auth path

Signing leaf L_1 :

Auth path = $[L_0, H_{23}]$

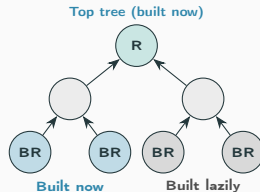
Verify: $H(H(L_0 || L_1) || H_{23}) = \text{Root}$

Multi-Layer Merkle Trees (Hypertrees)

Problem: Flat MSS KeyGen builds all 2^h OTS keys upfront — exponentially expensive for large h .

Idea: Stack two Merkle trees. The *top tree* is built immediately; each of its leaves certifies the root of a *bottom tree* that is built **lazily** only when needed.

- Total capacity: $N = 2^{h_{\text{top}} + h_{\text{bot}}}$ messages.
- Initial KeyGen: $O(2^{h_{\text{top}}} + 2^{h_{\text{bot}}})$ OTS keys.
- Signing produces *two* MSS signatures: one from the bottom tree, one from the top tree certifying the bottom root.
- This layering is the core idea behind **SPHINCS+** / **SLH-DSA** (NIST post-quantum standard).



Evaluation & Extensions

Evaluation Setup: How Messages Are Generated

Two distinct message generation strategies are used in the evaluation:

Security Simulation (Key-Reuse Attack)

- A mix of **fixed commands** and **random strings**:
 - "Attack at dawn", "Hold the line"...
 - `f"Random message {random.random()}"`
- 20 unique messages are signed with the *same* Lamport key.
- A fixed target ("Retreat at noon") tests if a forgery is possible.
- Repeated over **200 independent trials** for statistical validity.

Performance Benchmarking

- A single fixed string is used across all timing trials:

"Post-quantum cryptography is exciting"

- Each configuration is measured over **5 repetitions**; we report the *mean and variance* to capture both the typical time and its stability.
- The same message is reused across all parameter sweeps for a fair, controlled comparison.

Lamport Key-Reuse Attack

Threat model: A *passive* attacker observes k signatures made with the **same** Lamport key.

- Each signature reveals one block per position (bit 0 or 1).
- After k signatures, the probability a specific block is *still hidden* is $(1/2)^k$.

Forgery probability:

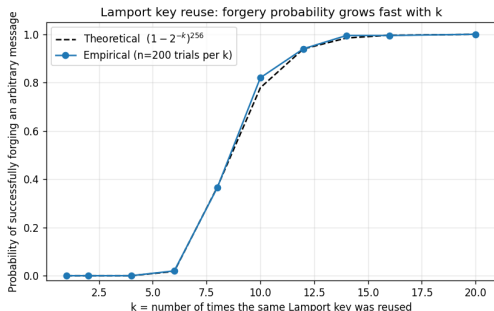
$$P_{\text{forge}}(k) = (1 - 2^{-k})^{256}$$

The attacker needs all 256 blocks matching the target message's hash bits. This converges to certainty exponentially.

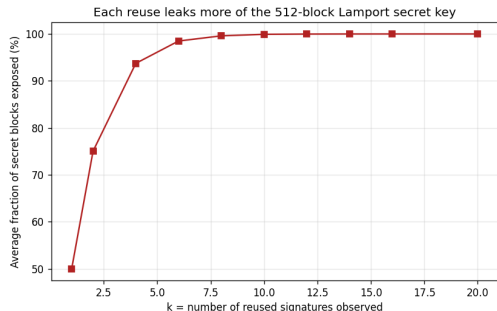
k	P_{forge}	SK Exposed
1	$\approx 0\%$	50%
5	0.03%	96.9%
8	38.5%	99.6%
10	78%	99.9%
12	94%	100%
15	99.8%	100%
20	100%	100%

Results from 200 independent trials per k .

Empirical Validation



Empirical vs. theoretical forgery rate.
Theory and experiment match closely.



Fraction of SK exposed vs. reuse count.
By $k=12$ the **complete** key is recovered.

OTS Performance Comparison

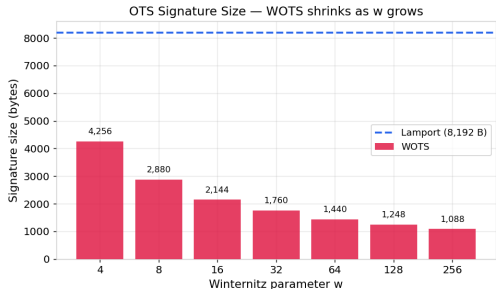
Scheme	KG (ms)	Sign (ms)	Verify (ms)	Sig (B)	PK (B)
Lamport	0.74 ± 0.013	0.02 ± 0.000	0.13 ± 0.002	8,192	16,384
WOTS $w=4$	0.29 ± 0.000	0.11 ± 0.000	0.10 ± 0.000	4,256	4,256
WOTS $w=16$	0.40 ± 0.000	0.19 ± 0.000	0.18 ± 0.000	2,144	2,144
WOTS $w=64$	1.06 ± 0.011	0.43 ± 0.000	0.55 ± 0.000	1,440	1,440
WOTS $w=256$	3.00 ± 0.006	1.56 ± 0.021	1.56 ± 0.014	1,088	1,088

Times reported as **mean** $\pm$ **variance** over 5 runs. The very small variances confirm the measurements are stable.

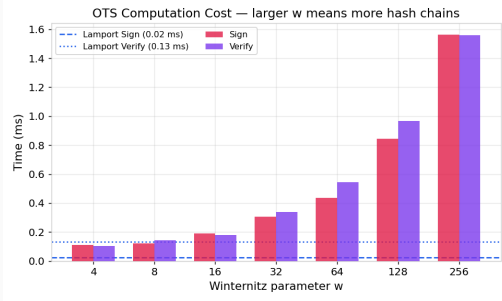
Key observations:

- Lamport signing is extremely fast (0.02 ms) — no hash computation, only copying blocks.
- WOTS $w=256$ is **7.5 $\times$ smaller** but much slower than Lamport.
- $w=16$ is the practical sweet spot — adopted by XMSS and SPHINCS+.

OTS: Size vs. Speed Trade-off



Signature size decreases *logarithmically* with w .



Computation time grows *linearly* — each chain is $w - 1$ hashes long.

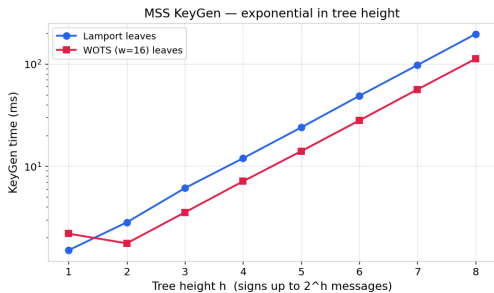
MSS Performance: Scaling with Tree Height

Height h	Messages	KG (ms)	Sign (ms)	Sig Size (B)
<i>Lamport leaves</i>				
4	16	11.96 ± 0.01	0.03 ± 0.00	24,640
6	64	48.73 ± 0.71	0.04 ± 0.00	24,704
8	256	196.28 ± 10.19	0.06 ± 0.00	24,768
<i>WOTS ($w=16$) leaves</i>				
4	16	7.11 ± 0.03	0.21 ± 0.00	4,420
6	64	27.99 ± 0.17	0.22 ± 0.00	4,484
8	256	112.17 ± 1.13	0.23 ± 0.00	4,548

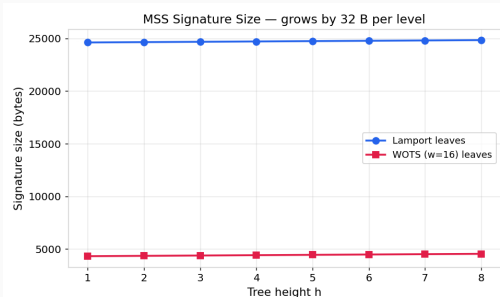
Times reported as **mean** $\pm$ **variance** over 5 runs.

- KeyGen: $O(2^h)$ — doubles with each additional level.
- Sign/Verify: **Constant** regardless of tree height.
- Signature overhead: only **+32 B per level** (one SHA-256 sibling).

MSS: KeyGen is the Bottleneck

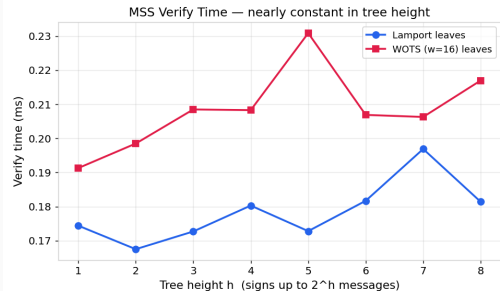
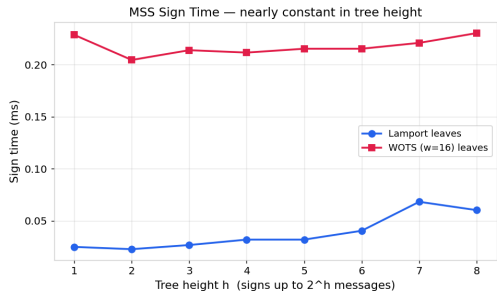


KeyGen scales as $O(2^h)$: the log-scale plot shows a linear trend.



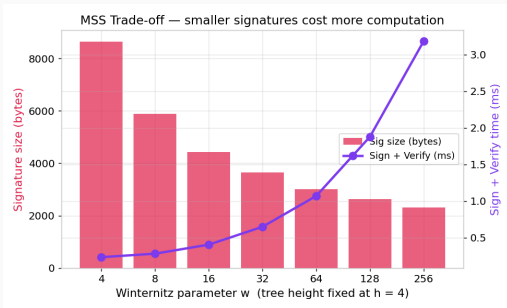
Signature size grows by just 32 B per tree level.

MSS: Sign & Verify are Constant-Time in h



The h additional hashes for the auth path are negligible compared to the underlying OTS operations.

Parameter Tuning: MSS with Varying w



Fixed: $h = 4$ (16 messages).

The *knee* near $w = 16$ marks the optimal balance:

- Below $w = 16$: rapid size improvement with modest time cost.
- Above $w = 16$: diminishing size returns, escalating time cost.

Recommendation

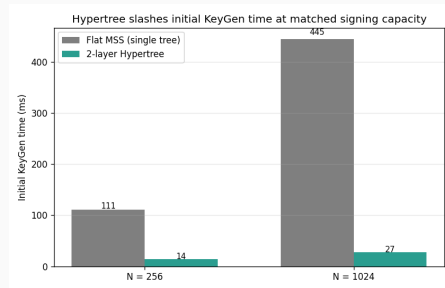
$w = 16$, $h = 4-8$

This matches the XMSS and SPHINCS+ standard configurations.

Multi-Layer Merkle Trees (Hypertrees): Performance

Scheme	N	KG (ms)	Sig (B)
Flat MSS, $h=8$	256	110.7 ± 0.6	4,548
Hypertree, $h_t = h_b = 4$	256	13.8 ± 0.01	8,876
Flat MSS, $h=10$	1024	444.9 ± 4.7	4,612
Hypertree, $h_t = h_b = 5$	1024	27.4 ± 0.4	8,940

Times: mean $\pm$ variance over 5 runs.



Result: $8.0\times$ faster KeyGen at $N=256$, $16.2\times$ faster at $N=1024$. Signature roughly doubles.

- We built the full pipeline from scratch using SHA-256: Lamport, WOTS, Merkle Signature Scheme, and a two-layer Hypertree.
- WOTS with $w = 16$ is the standard for a reason: four times smaller signatures than Lamport, with manageable speed cost.
- Merkle trees lift the one-time restriction: signing and verifying stay constant, and signatures barely grow with tree height.
- Hypertrees make the scheme practical at large scale: dramatically faster key generation at the cost of roughly doubling the signature size.
- Hash-based signatures rest on only one assumption — that SHA-256 is one-way — which is why they remain secure in the post-quantum era.

Thank You

Questions?

Aicha Slaitane — `aicha.slaitane@kaust.edu.sa`

Abdullah Algethami — `abdullah.algethami@kaust.edu.sa`